

AI + POSTGRESQL PERFORMANCE

AI won't fix your Postgres performance.

10x

It just makes you faster at finding the fix.

A DBA's field guide — what helps, and where AI lies.

THE TRAP

An LLM can't see your database

Text-to-SQL accuracy on 95 real, messy databases (BIRD benchmark).

GPT-4 (best case)



54.89%

Expert humans



92.96%

**~38
pts**

Grounding is everything — feed it your real stats.

Source: BIRD text-to-SQL benchmark (bird-bench.github.io), 2024.

THE RULE

Ground every prompt that matters

1

EXPLAIN (ANALYZE, BUFFERS)

Actual rows, timings, buffer hits — not estimates.

2

pg_stat_statements

calls, mean_exec_time, rows for the real workload.

3

version() + non-default settings

So advice matches your Postgres, not the internet's.

Withhold these and you're just autocompleting Stack Overflow.

WHERE AI HELPS

The value is wildly uneven

Triage & summarize

HIGH

Read query plans

HIGH

Index / rewrite advice

MEDIUM

Anomaly detection (ML)

HIGH

Internals & DDL

LOW

Medium = validate every suggestion. Low = keep a human on every lock.

Map of where AI helps across the Postgres performance workflow.

REAL EXAMPLE · COMPOSITE INDEX

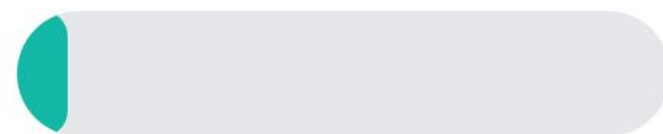
A missing composite index, found fast

Before — seq scan
+ sort



4,200 ms

After — composite
index



38 ms

~110x faster — validated with hypopg before
CREATE INDEX CONCURRENTLY.

Illustrative of a composite-index win; mechanism, SQL & tools are real.

AUTOVACUUM STARVATION

AI names the knob. You own the value.

Dead tuples climbing? AI nails the mechanism: vacuum triggers at $\text{threshold} + \text{scale_factor} \times \text{live rows}$.

0.2

Default `scale_factor` — too high on large, hot tables.

0.02

Your tuned per-table value — then watch `n_dead_tup`.

The mechanism is textbook. The value is workload-dependent — set it, watch, and tune from there.

Source: PostgreSQL docs — routine vacuuming.

SAFE WIRING

Connect AI to live stats, not the keys

[x]

Read-only role - SELECT on stats views, pg_monitor

[x]

Postgres MCP for live stats (safe-SQL mode)

[]

Never auto-apply DDL - a human runs every change

[x]

Validate with hypopg + a staging replica

Postgres MCP Pro defaults to read-only; humans own anything that locks.

Six steps, every slow query

1 Ground



2 Triage



3 Hypothesize



4 Validate



5 Apply



6 Measure

AI took the slow parts. It did not take the judgment.

[READ THE FULL FIELD GUIDE](#)

Grounded prompts. Validated changes.

That single habit is the whole difference between AI that helps and AI that pages you at 2 a.m.

sendtoshailash.github.io/blog/ai-postgresql-performance.html

Follow for the next one: grounding LLMs on live Postgres stats with MCP.

sendtoshailash.github.io - a DBA's field guide.